

BAB 7 Akses Fitur Perangkat

7.1 Tujuan Pembelajaran

- Memahami konsep akses fitur perangkat pada aplikasi mobile berbasis React Native (Expo).
- Menjelaskan mekanisme permission (perizinan) pada sistem Android dan iOS.
- Mengimplementasikan akses fitur perangkat seperti:
 - Kamera
 - Lokasi (GPS)
 - Audio
- Mengelola permission secara aman dan sesuai standar platform.
- Mengintegrasikan fitur perangkat ke dalam aplikasi secara modular dan terstruktur.

7.2 Dasar Teori

7.2.1 Konsep Akses Fitur Perangkat

Aplikasi mobile modern tidak hanya menampilkan informasi, tetapi juga berinteraksi langsung dengan perangkat keras (hardware) seperti:

- Kamera
- GPS
- Penyimpanan
- Mikrofon
- Sensor

Dalam pengembangan aplikasi menggunakan Expo (React Native), akses fitur perangkat dilakukan melalui Expo SDK, yang menyediakan API siap pakai untuk berbagai kebutuhan perangkat.

7.2.2 Permission (Perizinan Aplikasi)

Sistem operasi Android dan iOS menerapkan sistem keamanan berbasis permission, yaitu:

- Aplikasi harus meminta izin sebelum mengakses fitur sensitif.
- Pengguna dapat menyetujui atau menolak akses.
- Aplikasi harus menangani kondisi jika permission ditolak.

Contoh permission:

- Kamera → CAMERA
- Lokasi → LOCATION
- Media → MEDIA_LIBRARY

Pada Expo, permission biasanya diminta menggunakan method:

```
requestCameraPermissionsAsync()  
requestForegroundPermissionsAsync()  
requestMediaLibraryPermissionsAsync()
```

7.2.3 Library Expo untuk Akses Perangkat

Beberapa library utama yang digunakan:

Fitur	Library
-------	---------

Kamera	<i>expo-camera</i>
Lokasi	<i>expo-location</i>
Media	<i>expo-image-picker</i>
Penyimpanan	<i>expo-file-system</i>

Tabel 7.1 List Fitur dan Library

1. Expo Camera

Expo Camera (*expo-camera*) membahas *library* standar di ekosistem Expo untuk mengakses dan mengontrol kamera perangkat (depan dan belakang). Ini adalah komponen yang sangat sering digunakan untuk fitur seperti mengambil foto profil, mendokumentasikan bukti pengiriman, atau memindai QR Code.

A. Fitur Utama

- *Preview Kamera*: Menampilkan apa yang dilihat lensa kamera secara real-time di layar aplikasi.
- *Mengambil Foto & Video*: Menangkap gambar statis atau merekam video dengan kontrol kualitas.
- *Barcode/QR Code Scanning*: Secara otomatis mendeteksi dan membaca data dari barcode atau QR code yang masuk ke frame kamera.
- *Kontrol Kamera*: Mengatur zoom, flash (lampu kilat), white balance, dan fokus (auto/manual).
- *Multi-Kamera*: Mendukung peralihan antara kamera depan (selfie) dan belakang.

B. Instalasi & Konfigurasi

Untuk menggunakannya, harus menginstal package:

```
npx expo install expo-camera expo-image-picker
```

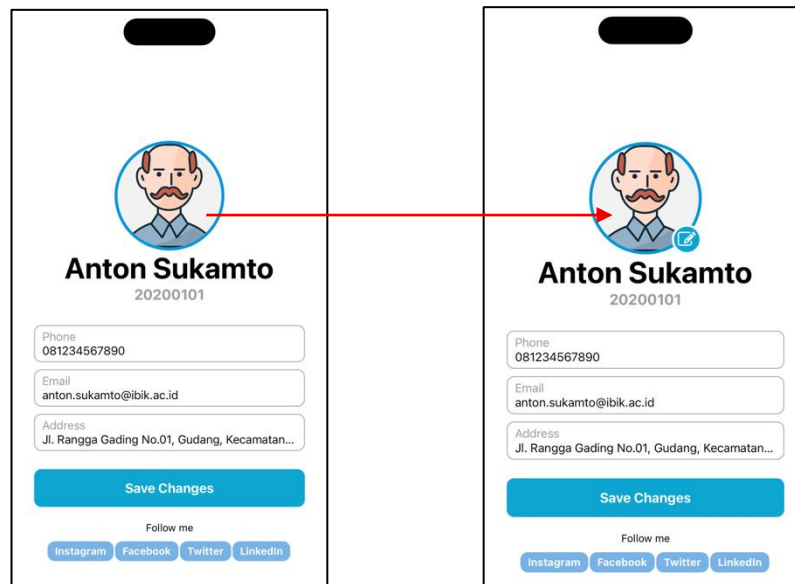
Expo Camera memerlukan izin privasi, dimana perlu menambahkannya di *app.json*:

```
{
  "expo": {
    "plugins": [
      [
        "expo-camera",
        {
          "cameraPermission": " Allow $(PRODUCT_NAME) to access your camera",
          "microphonePermission": " Allow $(PRODUCT_NAME) to access your microphone"
        }
      ]
    ]
  }
}
```

Contoh penggunaan:

Disini akan melanjutkan kode latihan3 pada BAB 3, dimana pada tampilan latihan3 pada tampilan avatar akan dimanfaatkan untuk mengganti update gambar foto profile, dengan menggunakan expo-camera dan expo-image-picker. Expo Camera digunakan untuk

menangkap hasil kamera dan akan ditampilkan pada element <Image/>, sedangkan Expo Image Picker digunakan untuk memilih gambar yang berasal dari galeri device.



Gambar 7.1 Tampilan Profile pada aplikasi

Mengubah element avatar dengan menambahkan JSX <TouchableOpacity /> sebagai kontainer <Image /> avatar.

Sebelum:

```
<Image
  source={require("../assets/avatars/avatar.png")}
  style={styles.headers.img_avatar}
  resizeMode="cover"
/>
```



Sesudah:

```

<TouchableOpacity>
  <View style={styles.avatarContainer}>
    <Image
      source={
        avatarUri
        ? { uri: avatarUri }
        : require("../assets/avatars/avatar.png")
      }
      style={styles.headers.img_avatar}
      resizeMode="cover"
    />
    <View style={styles.cameraIconOverlay}>
      <FontAwesome name="edit" size={24} color="#fff" />
    </View>
  </View>
</TouchableOpacity>

```



Tambahkan state management pada RFC tersebut:

```

const [avatarUri, setAvatarUri] = useState(null);
const [isCameraVisible, setIsCameraVisible] = useState(false);
const [permission, requestPermission] = useCameraPermissions();
const cameraRef = useRef(null);

```

Membuat sebuah modal atau pop up message yang diambil dari package react-native, modal ini akan tampil jika tombol edit avatar ditekan:

```

1  <Modal
2    visible={isCameraVisible}
3    animationType="slide"
4    onRequestClose={() => setIsCameraVisible(false)}
5  >
6    <View style={styles.cameraContainer}>
7      <CameraView
8        ref={cameraRef}
9        style={styles.camera}
10       facing="front"
11       mode="picture"
12     >
13       <View style={styles.cameraControls}>
14         <TouchableOpacity
15           style={styles.cancelButton}
16           onPress={() => setIsCameraVisible(false)}
17         >
18           <Text style={styles.buttonText}>Cancel</Text>
19         </TouchableOpacity>
20
21         <TouchableOpacity
22           style={styles.captureButton}
23           onPress={takePicture}
24         >
25           <View style={styles.captureButtonInner} />
26         </TouchableOpacity>
27
28         <View style={{ width: 80 }} />
29       </View>
30     </CameraView>
31   </View>
32 </Modal>

```

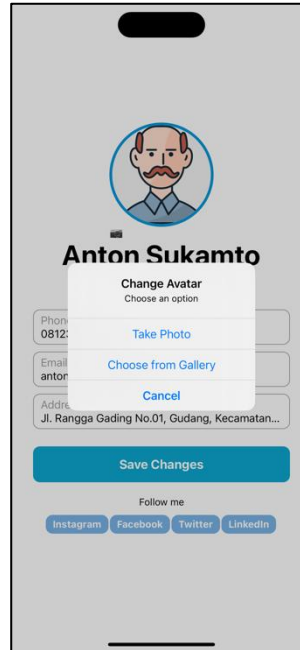
- Modal: Container fullscreen untuk kamera
- CameraView: Komponen kamera dari expo-camera
 - ref={cameraRef} → Referensi untuk takePicture
 - facing="front" → Kamera depan (selfie mode)
 - mode="picture" → Mode foto (bukan video)
- Controls: Tombol Cancel dan Capture

Membuat action handler untuk menampilkan modal tersebut dengan function bernama *showImagePickerOptions()*:

```

1  const showImagePickerOptions = () => {
2    Alert.alert(
3      "Change Avatar",
4      "Choose an option",
5      [
6        {
7          text: "Take Photo",
8          onPress: handleOpenCamera,
9        },
10       {
11         text: "Choose from Gallery",
12         onPress: pickImageFromGallery,
13       },
14       {
15         text: "Cancel",
16         style: "cancel",
17       },
18     ],
19     { cancelable: true },
20   );
21 };

```



Gambar 7.2 Tampilan Warning Edit Avatar

Fungsi: Menampilkan dialog pilihan untuk mengubah avatar

Pilihan:

- Take Photo → Buka kamera (handleOpenCamera)
- Choose from Gallery → Buka gallery (pickImageFromGallery)
- Cancel → Tutup dialog

Parameter Alert.alert:

- "Change Avatar" → Title dialog
- "Choose an option" → Message dialog
- Array buttons → Daftar tombol pilihan
- { cancelable: true } → Bisa ditutup dengan tap di luar dialog

Membuat action handler untuk membuka kamera pada function *handleOpenCamera()* dan menyimpan gambar yang telah diambil pada function *takePicture()*:

<pre>const handleOpenCamera = async () => { if (!permission) { const { status } = await requestPermission(); if (status !== "granted") { Alert.alert("Permission Denied", "Camera permission is required to take photos",); return; } } if (!permission.granted) { const { status } = await requestPermission(); if (status !== "granted") { Alert.alert("Permission Denied", "Camera permission is required to take photos",); return; } } setIsCameraVisible(true); };</pre>	<pre>const takePicture = async () => { if (cameraRef.current) { try { const photo = await cameraRef.current.takePictureAsync({ quality: 0.8, base64: false, }); setAvatarUri(photo.uri); setIsCameraVisible(false); } catch (error) { console.error("Error taking picture:", error); Alert.alert("Error", "Failed to take picture"); } } };</pre>
---	--

Selanjutnya ialah membuat action handler untuk mengambil gambar pada galeri menggunakan function `pickImageFromGallery()`:

```
const pickImageFromGallery = async () => {
  const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();

  if (status !== "granted") {
    Alert.alert(
      "Permission Denied",
      "Gallery permission is required to select photos",
    );
    return;
  }

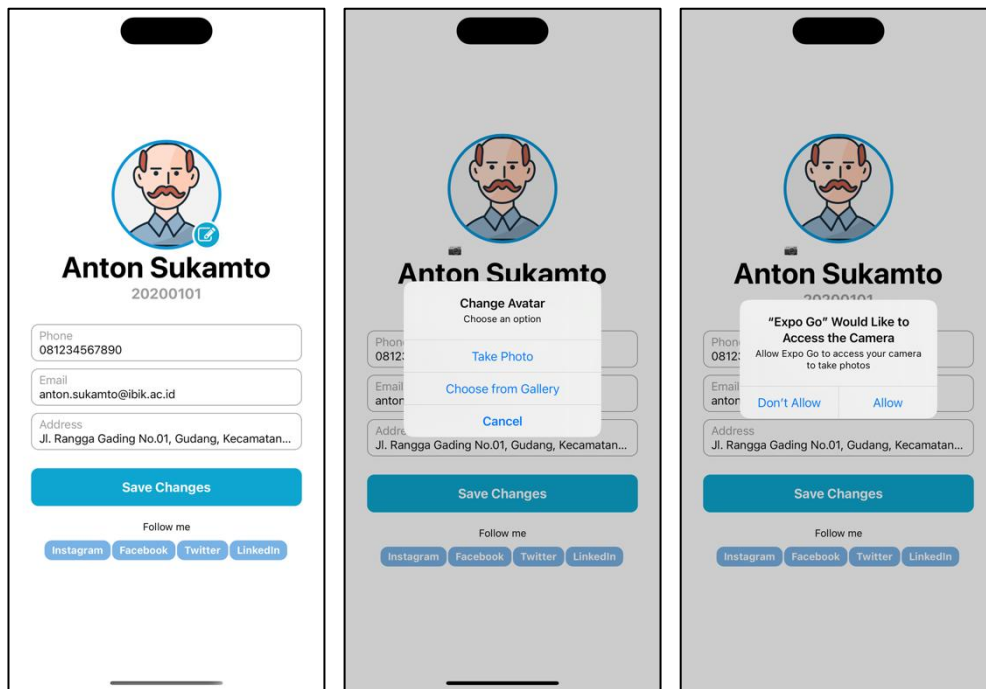
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ["images"],
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.8,
  });

  if (!result.canceled) {
    setAvatarUri(result.assets[0].uri);
  }
};
```

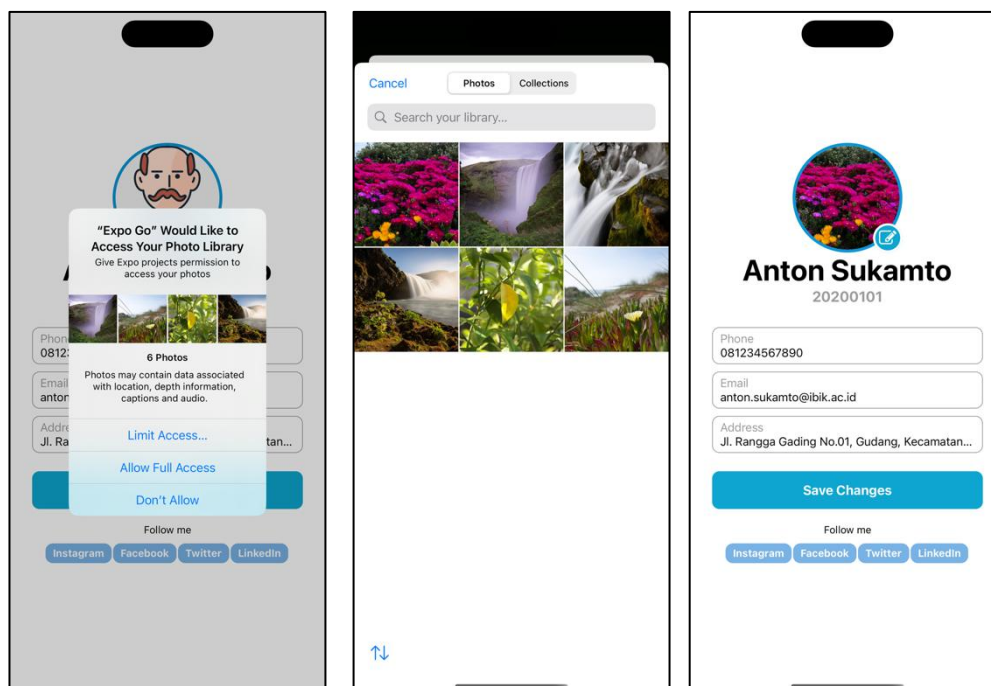
Terakhir ialah memasang handler pada `<TouchableOpacity />` untuk mengakses `showImagePickerOptions()` pada gambar avatar:

```
<TouchableOpacity onPress={showImagePickerOptions}>
  <View style={styles.avatarContainer}> ...
  </View>
</TouchableOpacity>
```

Maka hasil dari penggunaan expo-camera dan expo-image-picker seperti berikut:



Gambar 7.3 Tampilan Mengakses Kamera dengan memilih "Take Photo"



Gambar 7.4 Tampilan Mengakses Gallery dengan memilih "Choose form Gallery"

2. Expo-Location

expo-location ialah modul yang memungkinkan aplikasi React Native untuk:

- Mendapatkan koordinat GPS saat ini (Latitude/Longitude).
- Melacak pergerakan pengguna secara real-time.
- Mengonversi alamat menjadi koordinat (Geocoding) dan sebaliknya (Reverse Geocoding).

- Mengakses data arah mata angin (Kompas/Heading).
- Membuat fitur Geofencing (mendeteksi saat pengguna masuk/keluar area tertentu).

Fitur utama:

Fitur	Fungsi
Get Current Position	Mengambil lokasi pengguna satu kali saja (misal: untuk <i>tag</i> lokasi di foto).
Watch Position	Berlangganan (subscribe) ke pembaruan lokasi. Fungsi akan dipanggil setiap kali pengguna bergerak.
Background Location	Melacak lokasi bahkan saat aplikasi ditutup atau diminimalkan (memerlukan izin khusus).
Geocoding	Mengubah "Jakarta, Indonesia" menjadi koordinat, atau koordinat -6.2088, 106.8456 menjadi nama jalan.
Geofencing	Memicu <i>event</i> ketika pengguna memasuki atau meninggalkan radius lingkaran tertentu.

Tabel 7.2 Fitur pada Expo-Location

Instalasi & Konfigurasi

Untuk menggunakannya, harus menginstal package:

`npx expo install expo-location`

Konfigurasi Izin (Permissions):

- Ini adalah bagian paling krusial. Di Android dan iOS, wajib meminta izin pengguna.
- Foreground Permission: Izin mengakses lokasi saat aplikasi sedang dibuka.
- Background Permission: Izin mengakses lokasi saat aplikasi tertutup (lebih sulit didapatkan, terutama di Google Play Store).

Contoh penggunaan:

```

1 import * as Location from "expo-location";
2 import { useEffect, useState } from "react";
3 import { Text } from "react-native";
4 import { SafeAreaView } from "react-native-safe-area-context";
5
6 export default function Latihan7() {
7   const [location, setLocation] = useState(null);
8   const [address, setAddress] = useState(null);
9
10  useEffect(() => {
11    (async () => {
12      let { status } = await Location.requestForegroundPermissionsAsync();
13
14      if (status !== "granted") {
15        Alert.alert("Izin ditolak", "Aplikasi membutuhkan akses lokasi");
16        return;
17      }
18
19      let userLocation = await Location.getCurrentPositionAsync({});
20      setLocation(userLocation.coords);
21
22      // Reverse geocoding untuk mendapatkan alamat dari koordinat
23      let addressData = await Location.reverseGeocodeAsync({
24        latitude: userLocation.coords.latitude,
25        longitude: userLocation.coords.longitude,
26      });
27
28      if (addressData.length > 0) {
29        setAddress(addressData[0]);
30      }
31    })();
32  }, []);
33
34  return (
35    <SafeAreaView>
36      <Text style={{ padding: 5, backgroundColor: "brown", color: "white" }}>
37        Result Location: {JSON.stringify(location)}
38      </Text>
39      <Text>Latitude: {location?.latitude}</Text>
40      <Text>Longitude: {location?.longitude}</Text>
41      {address && (
42        <
43          <Text
44            style={{
45              fontWeight: "bold",
46              fontSize: 16,
47              marginTop: 15,
48              marginBottom: 5,
49            }}
50          >
51            Alamat:
52          </Text>
53          <Text>
54            Kota: {address.city || address.subregion || "Tidak tersedia"}
55          </Text>
56          <Text>Provinsi: {address.region || "Tidak tersedia"}</Text>
57          <Text>Negara: {address.country || "Tidak tersedia"}</Text>
58          <Text>Kode Pos: {address.postalCode || "Tidak tersedia"}</Text>
59          <Text>Jalan: {address.street || "Tidak tersedia"}</Text>
60          <Text>
61            Alamat Lengkap: {address.name || address.street}, {address.city},{ " "}
62            {address.region}, {address.country}
63          </Text>
64        </>
65      )}
66    </SafeAreaView>
67  );
68 }

```

Penjelasan:

A. State Management

`const [location, setLocation] = useState(null);`

```
const [address, setAddress] = useState(null);
```

- *location*: Digunakan untuk menyimpan data mentah koordinat (latitude, longitude, altitude, dll). Nilai awalnya null.
- *address*: Digunakan untuk menyimpan objek alamat hasil konversi dari koordinat. Nilai awalnya null.

B. Logika Utama (*useEffect*)

Bagian ini berjalan otomatis sekali saat komponen pertama kali dimuat (*mount*).

Langkah 1: Meminta izin

```
let { status } = await Location.requestForegroundPermissionsAsync();
if (status !== "granted") {
  Alert.alert("Izin ditolak", "Aplikasi membutuhkan akses lokasi");
  return;
}
```

Aplikasi meminta izin kepada user untuk mengakses lokasi saat aplikasi dibuka. Jika ditolak, fungsi berhenti (*return*).

Langkah 2: Mengambil Koordinat

```
let userLocation = await Location.getCurrentPositionAsync({});
setLocation(userLocation.coords);
```

Fungsi ini menyalakan GPS dan mengambil posisi saat ini. Hasilnya disimpan ke state *location*.

Langkah 3: Reverse Geocoding

```
let addressData = await Location.reverseGeocodeAsync({
  latitude: userLocation.coords.latitude,
  longitude: userLocation.coords.longitude,
});
if (addressData.length > 0) {
  setAddress(addressData[0]);
}
```

- *reverseGeocodeAsync*: Fungsi ini menerima input latitude & longitude, lalu mencari ke database peta (Google/Apple Maps) alamat apa yang ada di titik tersebut.
- Hasilnya berupa *array* (daftar kemungkinan alamat). Kode ini mengambil hasil pertama (*addressData[0]*) karena itu yang paling akurat.

C. Tampilan (UI)

Menampilkan Koordinat Mentah

```
<Text>
  Result Location: {JSON.stringify(location)}
</Text>
<Text>Latitude: {location?.latitude}</Text>
<Text>Longitude: {location?.longitude}</Text>
```

- *JSON.stringify*: Menampilkan seluruh objek data lokasi untuk tujuan debugging.

- `location?.latitude` dan `location?.longitude`: Tanda tanya (?) adalah Optional Chaining. Ini penting karena saat aplikasi baru dibuka, nilai location masih null. Tanpa tanda tanya, aplikasi akan crash/error karena mencoba membaca data dari null

Menampilkan Alamat (Conditional Rendering):

`{address && ( ... )}`

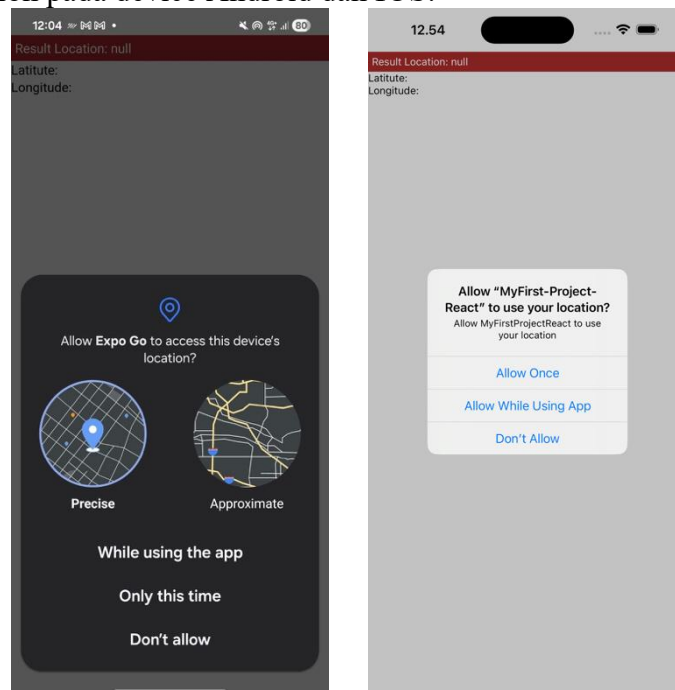
Blok kode ini hanya akan dirender atau ditampilkan JIKA state address sudah terisi data (tidak null).

Fallback Value:

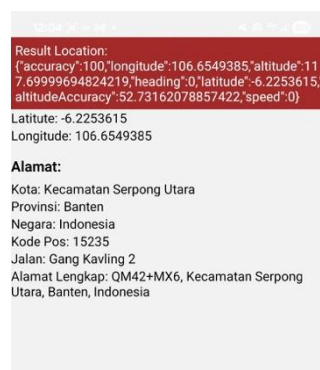
`<Text>Kota: {address.city || address.subregion || "Tidak tersedia"}</Text>`

- Struktur data alamat bisa berbeda-beda tergantung area (kadang city kosong tapi subregion ada).
- Kode ini menggunakan logika "ATAU" (||). Jika city kosong, coba ambil subregion. Jika kosong juga, tampilkan teks "Tidak tersedia".

Tampilan dari location pada device Android dan IOS:



Gambar 7.5 Tampilan Screen Permintaan Izin Android (Kiri) dan IOS (Kanan)



Gambar 7.6 Tampilan Screen Current Location

3. Expo Maps

expo-maps adalah library resmi dari Expo yang digunakan untuk menampilkan peta (map) di aplikasi React Native berbasis Expo.

Library ini memungkinkan aplikasi untuk:

- Menampilkan peta interaktif
- Menentukan lokasi (koordinat latitude & longitude)
- Menambahkan marker
- Mengatur zoom level
- Menampilkan lokasi tertentu

Instalasi package:

```
npx expo install expo-maps
```

Konfigurasi Platform pada `app.json`:

```
{
  "expo": {
    "plugins": [
      [
        "expo-maps",
        {
          "requestLocationPermission": true,
          "locationPermission": "Allow $(PRODUCT_NAME) to use your location"
        }
      ]
    ]
  }
}
```

Dalam penggunaan *expo-maps* secara default maps akan merender sesuai dengan operating sistem pada smartphone. Android akan menggunakan Google Maps sedangkan IOS menggunakan Apple Maps.

Konsep dasar penggunaan komponen utama yang digunakan ialah:

```
<MapView />
```

Komponen tersebut digunakan untuk menampilkan peta.

Contoh penggunaan menampilkan peta:

```

1  import { View } from "react-native";
2  import MapView from "expo-maps";
3
4  export default function MapScreen() {
5    return (
6      <View style={{ flex: 1 }}>
7        <MapView
8          style={{ flex: 1 }}
9          initialRegion={{
10             latitude: -6.606048383770911,
11             longitude: 106.79937370773561,
12             latitudeDelta: 0.05,
13             longitudeDelta: 0.05,
14           }}
15        />
16      </View>
17    );
18  }

```

Properti

Properti	Fungsi
latitude	Koordinat lintang
longitude	Koordinat bujur
latitudeDelta	Tingkat zoom vertikal
longitudeDelta	Tingkat zoom horizontal

Tabel 7.3 Properti Expo Maps

Dapat juga menggunakan library API dari masing-masing operating sistem dengan memanfaatkan package:

```
import { AppleMaps, GoogleMaps } from "expo-maps"
```

Namun dalam penggunaan AppleMaps dan GoogleMaps seperti contoh dibawah ini:

```

const LatihanMaps = () => {
  const [location, setLocation] = useState(null);
  const [cameraPosition, setCameraPosition] = useState(null);
  useEffect(() => {
    (async () => {
      let { status } = await Location.requestForegroundPermissionsAsync();

      if (status !== "granted") {
        Alert.alert("Izin ditolak", "Aplikasi membutuhkan akses lokasi");
        return;
      }

      let userLocation = await Location.getCurrentPositionAsync({});
      setLocation(userLocation.coords);
    })();
  }, []);

  useEffect(() => {
    if (location) {
      const { latitude, longitude } = location;
      const cameraPosition = {
        latitude,
        longitude,
        zoom: 18,
      };
      setCameraPosition(cameraPosition);
    }
  }, [location]);

  if (Platform.OS === "ios") {
    return (
      <AppleMaps.View
        style={{ flex: 1 }}
        showsCompass={true}
        cameraPosition={cameraPosition}
        markers={markersAddress}
        showsUserLocation={true}
      />
    );
  } else if (Platform.OS === "android") {
    return (
      <GoogleMaps.View
        style={{ flex: 1 }}
        cameraPosition={cameraPosition}
        markers={markersAddress}
        showsUserLocation={true}
        showsCompass={true}
      />
    );
  } else {
    return <Text>Maps are only available on Android and iOS</Text>;
  }
};

```

Penjelasan:

A. State Management

- *location*: Menyimpan koordinat lokasi pengguna (latitude & longitude)
- *cameraPosition*: Menyimpan posisi kamera peta yang akan ditampilkan

B. Mendapatkan Lokasi Pengguna (useEffect Pertama)

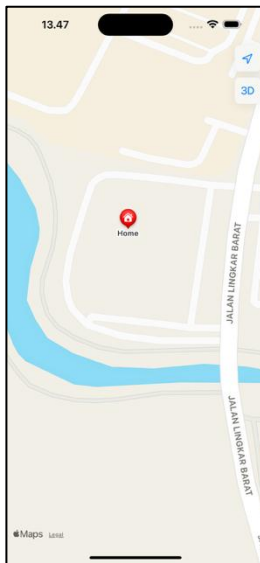
- Meminta izin akses lokasi dari pengguna
- Jika izin diberikan, mengambil koordinat lokasi pengguna saat ini
- Menyimpan koordinat ke *state location*

C. Mengatur Posisi Layar Peta (useEffect Kedua)

- Mendeteksi adanya perubahan nilai location
- Mengatur posisi layar peta berdasarkan lokasi pengguna saat ini
- Zoom level 18 memberikan tampilan yang cukup detail

D. Rendering Peta Berdasarkan Platform

- *Cross-Platform Support*: Menggunakan Apple Maps untuk iOS dan Google Maps untuk Android
- *User Location*: Menampilkan lokasi pengguna saat ini pada peta
- *Compass*: Menampilkan kompas untuk orientasi arah
- *Markers*: Menampilkan penanda lokasi (dari array *markers.Address*)
- *Camera Position*: Mengatur posisi dan zoom layar peta secara otomatis



Hasil yang diperoleh dari code tersebut akan mendeteksi jika smartphone dengan operating sistem IOS maka akan menampilkan layar peta menggunakan Apple Maps. Seperti gambar disamping ini, device yang digunakan ialah iPhone oleh karenanya akan menampilkan Apple Maps.

Namun jika ingin melakukan custome terhadap penggunaan map pada environment IOS, misal ingin menggunakan Google Map maka yang digunakan adalah GoogleMaps dan tidak menggunakan MapView.

Pada gambar disamping, titik lokasi marker bertuliskan "Home" mengartikan titik lokasi saat ini yang telah diolah atau diambil dari location.

7.3 Praktikum

Pemanfaatan fitur perangkat seperti lokasi dan peta menjadi salah satu komponen penting dalam meningkatkan pengalaman pengguna. Aplikasi tidak hanya menampilkan informasi secara statis, tetapi juga mampu memberikan layanan berbasis lokasi (*Location-Based Service*). Pada praktikum ini, kita akan melanjutkan pengembangan aplikasi *e-catalog* buku (*Readly+*) dengan menambahkan fitur Explore yang memungkinkan pengguna mencari toko buku terdekat melalui peta interaktif.

Komponen Explore adalah halaman utama untuk menampilkan peta interaktif yang menunjukkan lokasi toko buku di sekitar pengguna. Komponen ini mengintegrasikan *Google Maps* atau *Apple Maps*, *geolocation*, dan *bottom sheet* untuk memberikan pengalaman eksplorasi toko yang interaktif.

Langkah 1: Struktur Komponen

Explore (Main Component)

- MapView (Peta Interaktif)
- Header (Tombol Kembali)
- BottomSheet

└─ ListStores (Daftar Toko)

Pada screen Explore memiliki 3 buah komponen turunan. Dimulai pada komponen MapView:

RFC MapView

```
const MapView = ({ current_location }) => {
  const cameraPosition = {
    coordinates: {
      latitude: current_location?.latitude,
      longitude: current_location?.longitude,
    },
    zoom: 13,
  };

  const [mapError, setMapError] = useState(false);
  const [errorMessage, setErrorMessage] = useState("");

  if (Platform.OS === "web") {
    return (
      <View style={style_explore.map, style_explore.mapFallback}>
        <Ionicons name="map-outline" size={80} color={color_list.green} />
        <Text style={style_explore.mapFallbackText}>Map View Unavailable</Text>
        <Text style={style_explore.mapFallbackSubtext}>
          Maps are only available on Android and iOS devices
        </Text>
        <Text style={style_explore.mapFallbackHint}>
          Please run this app on a mobile device or emulator
        </Text>
      </View>
    );
  }

  if (Platform.OS === "ios") {
    return (
      <AppleMaps.View
        style={{ flex: 1 }}
        cameraPosition={cameraPosition}
        markers={markersAddress}
        showsUserLocation={true}
        showsCompass={true}
      />
    );
  } else if (Platform.OS === "android") {
    return (
      <GoogleMaps.View
        style={{ flex: 1 }}
        cameraPosition={cameraPosition}
        markers={markersAddress}
        showsUserLocation={true}
        showsCompass={true}
      />
    );
  }

  return (
    <View style={style_explore.map, style_explore.mapFallback}>
      <Ionicons name="map-outline" size={80} color={color_list.green} />
      <Text style={style_explore.mapFallbackText}>Map View Unavailable</Text>
      <Text style={style_explore.mapFallbackSubtext}>Unsupported platform</Text>
    </View>
  );
};
```

RFC Header

```
const Header = () => {
  const router = useRouter();
  return (
    <SafeAreaView style={style_explore.headerContainer} edges={["top"]} >
      <View style={style_explore.headerContent}>
        <TouchableOpacity onPress={() => router.push("/main-apps")} >
          <Ionicons name="arrow-back" size={24} color="black" />
        </TouchableOpacity>
      </View>
    </SafeAreaView>
  );
};
```

RFC ListStores

```
const ListStores = ({stores}) => {
  return (
    <View>
      {stores.map((store, index) => (
        <View key={index}>
          <View>
            <AntDesign name="shopping" size={24} color={"green"} />
          </View>
          <View>
            <Text>
              {store?.title}
            </Text>
            <Text>
              <AntDesign name="star" size={18} color={"orange"} /> 4
            </Text>
            <Text>Open &middot; Closes 10.00 pm &middot; (021) 1234-5678</Text>
          </View>
        </View>
      ))}
    </View>
  );
};
```

RFC Explore (Parent Screen)

```
import { Ionicons } from "@expo/vector-icons";
import AntDesign from "@expo/vector-icons/AntDesign";
import BottomSheet, {
  BottomSheetBackdrop,
  BottomSheetScrollView,
} from "@gorhom/bottom-sheet";
import * as Location from "expo-location";
import { AppleMaps, GoogleMaps } from "expo-maps";
import { useRouter } from "expo-router";
import { useEffect, useMemo, useRef, useState } from "react";
import { Platform, StyleSheet, Text, View } from "react-native";
import { GestureHandlerRootView } from "react-native-gesture-handler";
import { SafeAreaView } from "react-native-safe-area-context";
```

```

export default function Explore() {
  const bottomSheetRef = useRef(null);
  const [location, setLocation] = useState(null);
  const [address, setAddress] = useState(null);

  const snapPoints = useMemo(() => ["35%", "50%", "90%"], []);
  const handleSheetChange = (index) => {
    if (index === -1) {
      bottomSheetRef.current.snapToIndex(0);
    }
  };

  useEffect(() => {
    (async () => {
      let { status } = await Location.requestForegroundPermissionsAsync();

      if (status !== "granted") {
        Alert.alert("Izin ditolak", "Aplikasi membutuhkan akses lokasi");
        return;
      }

      let userLocation = await Location.getCurrentPositionAsync({});
      setLocation(userLocation.coords);

      let addressData = await Location.reverseGeocodeAsync({
        latitude: userLocation.coords.latitude,
        longitude: userLocation.coords.longitude,
      });

      if (addressData.length > 0) {
        setAddress(addressData[0]);
      }
    })();
  }, []);
}

```

```

return (
  <View style={style_explore.container}>
    <GestureHandlerRootView>
      <MapView current_location={location} />
      <Header />
      <BottomSheet
        ref={bottomSheetRef}
        index={0}
        enablePanDownToClose
        onChange={handleSheetChange}
        snapPoints={snapPoints}
        backdropComponent={(props) => (
          <BottomSheetBackdrop
            {...props}
            opacity={0.5}
            appearsOnIndex={1}
            disappearsOnIndex={0}
          />
        )}
        backgroundStyle={{
          borderTopLeftRadius: 20,
          borderTopRightRadius: 20,
          backgroundColor: "white",
        }}
      >
        <BottomSheetScrollView>
          <Text>Explore Store</Text>
          {address && (
            <Text style={style_explore.subtitle}>
              Location:{" "}
              {(address?.city || address?.name || "-") +
                ", " +
                (address?.subregion || address?.region || "-")}
            </Text>
          )}

          <View style={{ marginTop: 20 }}>
            <ListStores />
          </View>
        </BottomSheetScrollView>
      </BottomSheet>
    </GestureHandlerRootView>
  </View>
);
}

```

Penjelasan

A. State Management

- Bottom Sheet States
 - *bottomSheetRef*: Reference untuk kontrol bottom sheet
 - *snapPoints*: Tinggi bottom sheet (35%, 50%, 90% dari layar)
 - *sheetIndex*: Index snap point yang aktif (0 = 35%, 1 = 50%, 2 = 90%)
 - *function handleSheetChange()*: Jika user swipe down hingga close (index === -1), kembalikan ke posisi awal (35%). Jika tidak, update index sesuai posisi sheet
- Location State
 - *location*: Menyimpan koordinat GPS pengguna (latitude, longitude)
 - *address*: Menyimpan alamat lengkap hasil reverse geocoding

B. Location Tracking

- Request Permission: Minta izin akses lokasi dari user
- Get GPS Coordinates: Ambil koordinat GPS saat ini
- Reverse Geocoding: Konversi koordinat menjadi alamat (nama kota, region, dll)

- Update State: Simpan lokasi dan alamat ke state

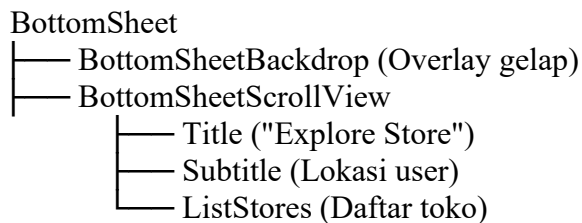
C. MapView Component

- Jika `current_location` ada, gunakan lokasi user
- Jika tidak, gunakan default

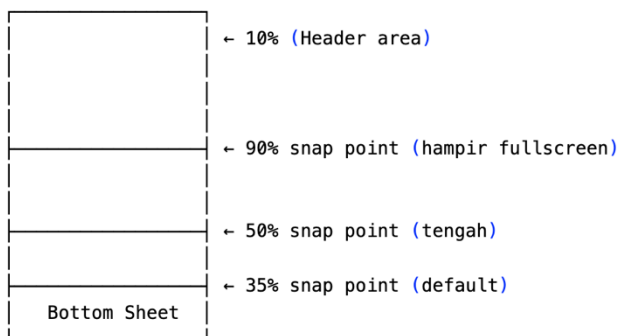
D. Bottom Sheet Content

Bottom Sheet adalah komponen UI yang muncul dari bawah layar, menampilkan informasi toko buku di sekitar lokasi pengguna. Komponen ini menggunakan library `@gorhom/bottom-sheet` untuk interaksi yang smooth dan responsif.

Struktur Bottom Sheet



Snap Points Configuration



```
const snapPoints = useMemo(() => ["35%", "50%", "90%"], []);
```

`useMemo`: Mencegah re-render yang tidak perlu
 ["35%", "50%", "90%"]: 3 tinggi yang bisa dipilih user
35%: Posisi awal (preview mode)
50%: Posisi tengah (comfortable reading)
90%: Posisi maksimal (full list view)

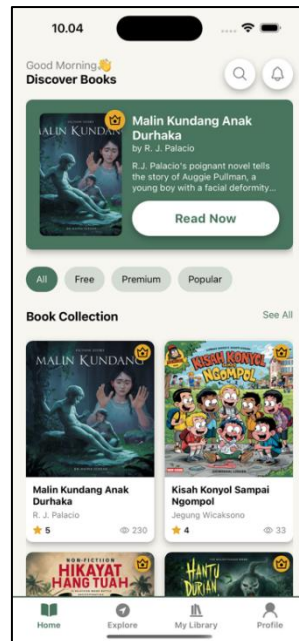
Penjelasan Props:

Prop	Value	Fungsi
<code>ref</code>	<code>bottomSheetRef</code>	Reference untuk kontrol programatik
<code>index</code>	0	Mulai di snap point pertama (35%)
<code>enablePanDownToClose</code>	true	User bisa swipe down untuk close
<code>onChange</code>	<code>handleSheetChange</code>	Callback saat posisi berubah
<code>snapPoints</code>	<code>["35%", "50%", "90%"]</code>	Tinggi yang tersedia
<code>backdropComponent</code>	Custom component	Overlay gelap di belakang sheet
<code>backgroundStyle</code>	Style object	Styling untuk background sheet

Tabel 7.4 Props pada React yang digunakan

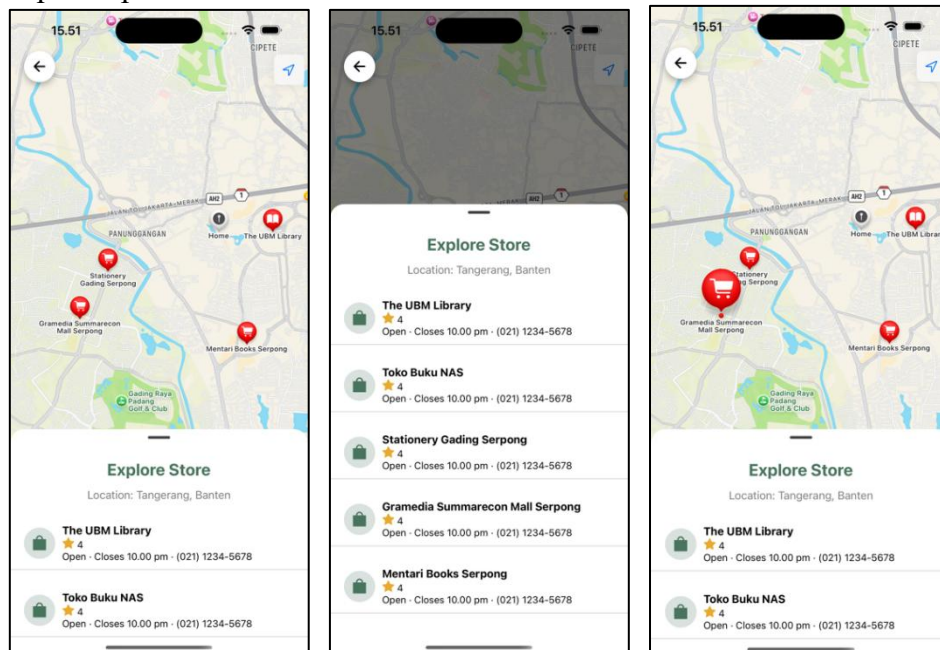
Langkah 2: Output tampilan maps pada IOS dan Android

Dari HomeScreen untuk mengakses halaman Explore diakase melalui Bottom Navigation. Ketika berpindah ke halaman Explore maka screen tersebut akan keluar dari Tab Navigation dan berpindah kedalam Stack Navigation.



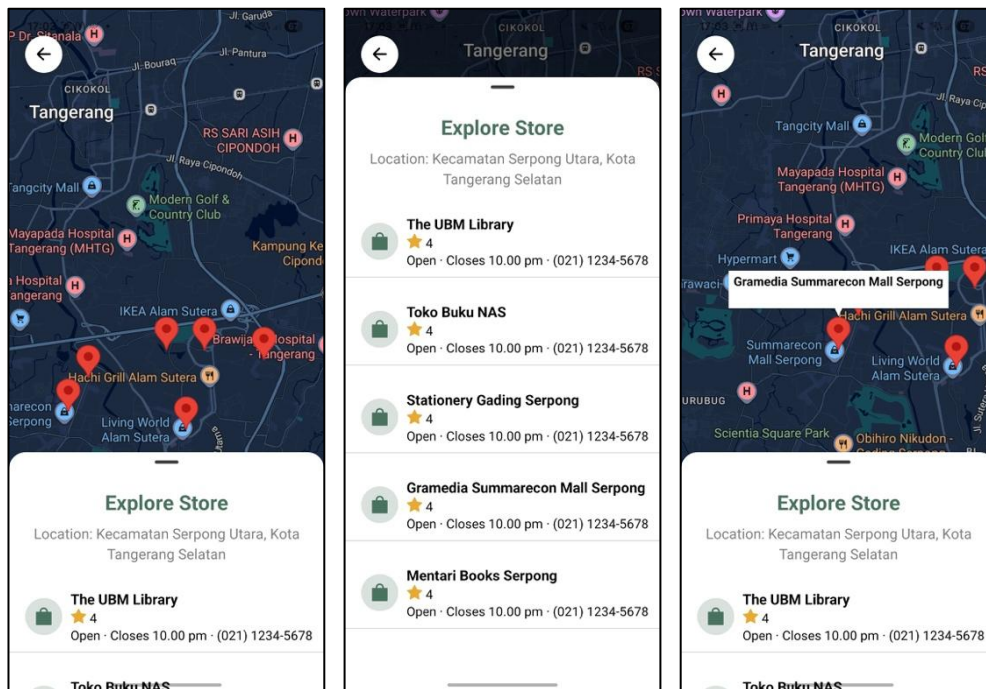
Gambar 7.7 Tampilan Output Program Utama

Tampilan Explorer pada IOS



Gambar 7.8 Tampilan Output Program Maps pada IOS

Tampilan Explorer pada Android



Gambar 7.9 Tampilan Output Program Maps pada Android

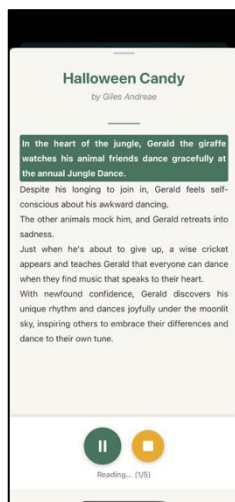
7.4 Tugas Latihan Praktikum

Pada tugas praktikum ini, akan mengimplementasikan pemanfaatan fitur perangkat (*device features*) dalam aplikasi mobile yang telah dikembangkan sebelumnya. Fokus utama pada bab ini adalah bagaimana aplikasi dapat berinteraksi langsung dengan kemampuan perangkat seperti kamera, media library, dan sistem pemindaian QR Code.

Mahasiswa diminta untuk mengembangkan dua fitur utama, yaitu:

Latihan 1: Membuat Halaman Profile yang dapat memperbaharui data pribadi dan avatar pengguna.

Latihan 2: Membuat fitur QR Scanner untuk membaca QR Code sebuah buku dan menampilkan detailnya



Latihan 3: Mengimplementasikan fitur perangkat output audio (speaker) yang tidak perlu meminta izin akses perangkat. Buatlah seperti detail buku seperti dibawah ini:

Pada halaman detail buku ini, dapat diubah menjadi audio-book dengan memanfaatkan teknologi text-to-speech.

Manfaatkanlah library package expo-speech dalam mengimplementasikan hal ini. Dimana mengubah sebuah text menjadi sebuah audio dan dikeluarkan melalui speaker device.

Dihalaman ini audio-book dapat di play, pause ataupun stop. Dan memberikan highlight pada text yang sedang dibacakan atau dikonversikan kedalam audio.

Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam *project react expo* yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository **"PPB-Praktikum-NPM"**, namun commit dengan nama **"Tugas Praktikum-7"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).